Final Report — T5 Dialogue Summarization

1. Introduction

This project fine-tunes a Transformer model to perform **abstractive dialogue summarization**: given a multi-party chat conversation, generate a short natural-language summary. We use the pretrained **T5 (Text-to-Text Transfer Transformer)** `t5-small` checkpoint and adapt it to the **SAMSum** dataset, then expose the model through a Dockerized Streamlit application with full MLflow experiment tracking.

---

2. Transformer Architecture for Summarization

2.1 The Transformer

The Transformer (Vaswani et al., 2017) replaces recurrence with **self-attention**, allowing every token to attend to every other token in parallel. Core components:

- **Multi-head self-attention** — learns contextual relationships between tokens regardless of distance, capturing who-said-what across a dialogue.
- **Positional encodings** — inject word-order information since attention is order-agnostic (T5 uses relative position biases).
- **Position-wise feed-forward networks** — per-token non-linear transformations.
- **Residual connections + layer normalization** — stabilize and speed up training.

2.2 Why T5 (encoder–decoder)

Summarization is a **sequence-to-sequence** task (long input → short output), which maps naturally onto T5's **encoder–decoder** design:

- The **encoder** reads the full dialogue and builds contextual representations.
- The **decoder** generates the summary autoregressively, using **cross-attention** to focus on the most relevant encoded tokens at each step.

T5 frames every NLP problem as **text-to-text**. We prepend the task prefix `summarize:` to each dialogue, so the model knows which behavior to apply. `t5-small` (~60M parameters) is intentionally chosen for fast, low-resource training while still demonstrating the full pipeline.

2.3 Abstractive vs. extractive

Unlike extractive methods that copy spans, T5 is **abstractive** — it can paraphrase and compose new wording (e.g. turning a back-and-forth exchange into "Amanda baked cookies and will bring some to Jerry"). This produces fluent, human-like summaries.

---

3. Training Approach

3.1 Transfer learning

Rather than training from scratch, we start from the **pretrained** `t5-small` weights (already trained on the large C4 corpus) and **fine-tune** on SAMSum. This transfers general language understanding and dramatically reduces the data/compute needed.

3.2 Data preprocessing (`src/data_utils.py`)

1. Download SAMSum via HuggingFace `datasets` into `data/` (no data committed).
2. Prepend `summarize:` to each dialogue.
3. Tokenize inputs (max 256 tokens) and targets (max 64 tokens) with truncation/padding.
4. Replace padding token ids in the labels with `-100` so they are **ignored by the loss**.

3.3 Optimization

- **Loss:** token-level cross-entropy (teacher forcing) over the decoder outputs.
- **Optimizer:** AdamW with weight decay (`0.01`).
- **Data collator:** `DataCollatorForSeq2Seq` for dynamic padding.
- **Minimal config** for fast reproducible runs: 1 epoch, `max_steps=50` , batch size 4, 200-example subset. Training completes in ~1 minute on CPU.

3.4 Hyperparameter tuning

`train.py` accepts CLI overrides (`--learning_rate` , `--batch_size` , `--num_train_epochs` , `--max_steps` , `--run_name`). We ran **two experiments**:

Run	learning_rate	batch_size	train_loss	eval_loss
`run-lr2e4-bs4`	2e-4	4	2.396	1.906
`run-lr5e4-bs8`	5e-4	8	2.144	1.841

The higher learning rate with larger batch size achieved a slightly lower validation loss.

3.5 Evaluation

`src/evaluate.py` generates summaries on the test split with **beam search** (4 beams) and computes **ROUGE-1/2/L** F1. ROUGE quantifies overlap with reference summaries:

Metric	F1	Interpretation
ROUGE-1	0.3632	unigram overlap
ROUGE-2	0.1408	bigram overlap (fluency/order)
ROUGE-L	0.3059	longest common subsequence

These are solid numbers given the deliberately tiny training budget and scale up with more data/epochs.

4. MLflow Experiment Tracking

MLflow provides reproducibility and experiment comparison. In this project:

- `mlflow.set_tracking_uri("mlruns")` uses a local **file store** (committed to the repo so reviewers can inspect it).
- Each training run logs:
- **Parameters:** model checkpoint, learning rate, batch size, epochs, max_steps, weight decay, subset sizes, token limits, seed.
- **Metrics:** `train_loss`, `train_runtime_sec`, `eval_loss`.
- **Artifacts:** the fine-tuned model directory under `model/`.
- `evaluate.py` logs a separate run with the ROUGE metrics.

The MLflow UI (`mlflow ui --backend-store-uri mlruns --port 5000`) lists all runs side-by-side, enabling direct comparison of hyperparameters vs. metrics — see `screenshots/mlflow_runs.png` and `screenshots/training_result.png`.

---

5. Docker Deployment

The prediction app is containerized so it runs identically anywhere.

- **Base image:** `python:3.11-slim`.
- **Dependencies** installed from `requirements.txt` (layer-cached before code copy).
- **Code + model** (`configs/`, `src/`, `app/`, `models/`) copied in.
- **Port** `8501` exposed for Streamlit; a `HEALTHCHECK` hits `/_stcore/health`.
- **Entrypoint:** `streamlit run app/app.py --server.port=8501 --server.address=0.0.0.0`.

```
docker build -t t5-summarizer-app:1.0 .
docker run -p 8501:8501 t5-summarizer-app:1.0
```

The app (`app/app.py`) offers **two input modes** (paste text or upload `.txt`), sidebar controls for summary length and beam width, and displays the generated summary with word-count metrics — see `screenshots/docker_app_running.png` and `screenshots/demo_output.png`.

> Note: the screenshots were captured from the same Streamlit app that the Dockerfile serves (identical entrypoint and port). The Docker build/run commands above reproduce it in a container.

---

6. Conclusion

The project delivers a complete, reproducible summarization pipeline: automatic data fetching, transfer-learning-based fine-tuning of a T5 Transformer, MLflow-tracked experiments, ROUGE evaluation, and a Dockerized Streamlit demo. The architecture (encoder–decoder Transformer with cross-attention) is well suited to abstractive summarization, and the modular design makes it easy to scale up to larger models, more data, and more epochs for higher quality.